

✅ SPRING CORE & SPRING BOOT

1 2 Spring Fundamentals (Must Know 🚨)

- Spring Framework overview
- IoC (Inversion of Control)
- DI (Dependency Injection)
- Who performs DI (Spring Container)
- Spring Context
- BeanFactory vs ApplicationContext

1 3 Spring Beans & Scopes (High Priority 🚨)

- What is a Spring Bean
- Bean lifecycle (basic)
- Bean scopes:
 - o Singleton
 - o Prototype
 - o Request
 - o Session
 - o Application (Global scope)
 - o WebSocket
- Default bean scope

1 4 Dependency Injection Annotations

- @Autowired
- Multiple beans of same type issue
- NoUniqueBeanDefinitionException
- @Qualifier
- @Primary
- Constructor vs Setter vs Field injection

1 5 Spring Boot Annotations (Very Important 🚨)

- @SpringBootApplication
 - o @Configuration
 - o @EnableAutoConfiguration
 - o @ComponentScan
- @EnableAutoConfiguration
- Custom auto configuration
- Excluding auto configurations
- @Component vs @Service vs @Repository
- @Bean vs @Component

- Stereotype annotations
 - Layered architecture
-

1 6 Spring Boot Configuration

- application.properties / application.yml
 - pom.xml contents
 - Changing server port (8080 → 8090)
 - Profiles (@Profile, spring.profiles.active)
 - Default embedded server (Tomcat)
 - Jetty & Undertow support
 - Standalone Spring Boot application
 - Advantages & disadvantages of Spring Boot
-

1 7 REST API & Web (High Priority 🚨)

- REST principles
 - @Controller vs @RestController
 - Default return type in @RestController
 - Why response body is required
 - REST vs SOAP
 - Safe vs Idempotent requests
 - HTTP methods (GET, POST, PUT, DELETE)
 - CRUD mapping annotations
 - @RequestMapping
 - @PathVariable
 - @RequestParam
 - @RequestBody
 - consumes & produces
 - Returning XML from REST API
-

1 8 Exception Handling in Spring Boot (Very Important 🚨)

- @ExceptionHandler
 - @ControllerAdvice
 - Global exception handling
 - REST exception handling flow
-

1 9 Spring Data JPA & Hibernate

- Hibernate advantages
- Hibernate vs JPA
- Lazy loading
- Caching in Hibernate
- JPA annotations:
 - o @Entity

- o @Table
 - o @Id
 - Database connectivity in Spring Boot
 - PreparedStatement vs Statement
 - CrudRepository vs JpaRepository
-

2 0 Actuator & Monitoring

- Spring Boot Actuator
 - Actuator endpoints:
 - o /health
 - o /metrics
 - o /info
 - o /beans
 - o /env
-

2 1 Microservices Architecture (Very Important 🚨)

- Monolithic vs Microservices
 - Advantages & disadvantages
 - Load Balancing
 - Circuit Breaker
 - Role of Spring Boot in Microservices
 - Spring Cloud overview
-

2 2 Security & Async

- Spring Security basics
 - @Async annotation
 - When & why async is needed
-

2 3 External Integrations

- Interacting with third party APIs
- Accessing third party URLs
 - o RestTemplate
 - o WebClient explain each topic in details one at a time with use case why it use when to use or when to not , mistakes make by developer tips for interview start by one part at time as i want deep explanation of each with example also



Great—this is the **right way to prepare** for Spring Boot interviews: structured + deep + use-case oriented.

We'll go **one section at a time**, with:

- ✓ Concept clarity
 - ✓ Real-world use case
 - ✓ When to use / NOT use
 - ✓ Common mistakes
 - ✓ Interview tips
 - ✓ Code examples
-

✓ PART 1 2 — SPRING FUNDAMENTALS (Must Know 🚨)

◆ 1. What is Spring Framework?

✓ Definition

The **Spring Framework** is a **lightweight, modular Java framework** used to build **enterprise-level applications** by simplifying:

- Object creation
 - Dependency management
 - Configuration
 - Integration
-

✓ Core Idea

Spring removes **tight coupling** between classes using:

👉 **IoC (Inversion of Control)**

👉 **DI (Dependency Injection)**

✓ Why Spring is used?

Without Spring:

```
<> Java 📄  
  
class Service {  
    Repository repo = new Repository(); // tightly coupled  
}
```

Problems:

- Hard to test
 - Hard to change implementation
 - Not scalable
-

With Spring:

</> Java



```
class Service {  
    private Repository repo;  
  
    public Service(Repository repo) {  
        this.repo = repo;  
    }  
}
```

👉 Spring injects the dependency → **loosely coupled system**

✓ Real-world Use Case

Think of:

- E-commerce (Amazon-like)
- Banking systems
- Microservices APIs

Spring helps:

- Manage thousands of objects (beans)
 - Handle transactions
 - Connect databases
 - Build REST APIs
-

✓ When to Use Spring?

Use Spring when:

- Building **enterprise apps**
 - Need **scalability**
 - Need **dependency management**
 - Building **REST APIs / Microservices**
-

✗ When NOT to Use?

Avoid Spring when:

- Small scripts or utilities
 - Simple Java programs
 - Performance-critical low-level systems (where overhead matters)
-

⚠ Common Mistakes

- ✗ Treating Spring as just annotations
 - ✗ Not understanding IoC (biggest mistake 🚨)
 - ✗ Overusing Spring where simple Java is enough
-

🎯 Interview Tip

If asked:

"What is Spring?"

👉 Answer like this:

"Spring is a lightweight framework that provides dependency injection and inversion of control to build loosely coupled, scalable Java applications."

◆ 2. IoC (Inversion of Control)

✓ Definition

IoC means:

👉 Spring controls object creation instead of you

✓ Traditional Flow (Without IoC)

</> Java

```
Service s = new Service(); // YOU create object
```



✓ With IoC

 Java



```
@Service  
class Service { }
```

Spring:

- Creates object
 - Manages lifecycle
 - Injects dependencies
-

✓ Real-world Analogy

Without IoC:

👉 You cook food yourself

With IoC:

👉 You order food → restaurant (Spring) handles everything

✓ Why IoC?

- Decouples code
 - Improves testing
 - Centralized control
-

✗ When NOT useful?

- Very small applications
 - No dependency complexity
-

⚠ Common Mistakes

- Thinking IoC = DI ✗
👉 DI is a way to achieve IoC
-

🎯 Interview Tip

👉 Key line:

"IoC shifts control of object creation from the programmer to the Spring container."

◆ 3. Dependency Injection (DI)

✓ Definition

DI means:

👉 Providing dependencies from outside instead of creating inside the class

✓ Types of DI

1. Constructor Injection ✓ (BEST)

⟨⟩ Java



```
@Service
class Service {
    private final Repo repo;

    public Service(Repo repo) {
        this.repo = repo;
    }
}
```

2. Setter Injection

⟨⟩ Java



```
@Service
class Service {
    private Repo repo;

    @Autowired
    public void setRepo(Repo repo) {
        this.repo = repo;
    }
}
```

3. Field Injection ❌ (avoid in interviews)

</> Java



```
@Autowired  
private Repo repo;
```

✓ Real-world Use Case

- Service needs Repository
- Controller needs Service
- Payment service needs gateway

✓ When to Use?

- Always use DI in Spring
- Prefer **Constructor Injection**

❌ When NOT to Use?

- Avoid Field Injection (testability issue)

⚠️ Common Mistakes

- ❌ Using Field Injection
- ❌ Not making dependencies `final`
- ❌ Circular dependencies

🎯 Interview Tip

👉 Best answer:

"Constructor injection is preferred because it ensures immutability and makes testing easier."

◆ 4. Who Performs DI?

✓ Answer

👉 Spring Container

✓ What is Spring Container?

It is responsible for:

- Creating objects (beans)
 - Injecting dependencies
 - Managing lifecycle
-

✓ Types of Containers

Container	Description
BeanFactory	Basic container
ApplicationContext	Advanced container (used in real apps)

✓ Real-world Flow

1. Spring scans classes
 2. Creates beans
 3. Injects dependencies
 4. Manages lifecycle
-

🎯 Interview Tip

"The Spring container manages beans and performs dependency injection using ApplicationContext."

◆ 5. Spring Context

✓ Definition

👉 Spring Context = **Container + Configuration + Beans**

It is represented by:

```
</> Java
```



```
ApplicationContext context;
```

✓ Responsibilities

- Bean creation
- Dependency injection
- Configuration loading
- Event handling

✓ Example

```
</> Java
```



```
ApplicationContext context =  
    new AnnotationConfigApplicationContext(AppConfig.class);  
  
Service s = context.getBean(Service.class);
```

✓ Real-world Use Case

- Bootstrapping entire application
- Managing services in backend

⚠ Common Mistakes

- Confusing context with bean ✖
- Not understanding lifecycle

🎯 Interview Tip

"ApplicationContext is the central interface for accessing Spring container and its beans."

◆ 6. BeanFactory vs ApplicationContext

✓ Key Difference Table

Feature	BeanFactory	ApplicationContext
Initialization	Lazy	Eager
Features	Basic	Advanced
Event handling	✗	✓
AOP support	✗	✓
Recommended	✗	✓

✓ Example

✎ Java



```
// BeanFactory
BeanFactory factory = new XmlBeanFactory(...);

// ApplicationContext
ApplicationContext context =
    new ClassPathXmlApplicationContext("config.xml");
```

✓ When to Use?

- ApplicationContext → Always use in real apps
- BeanFactory → Rare, memory-constrained systems

⚠ Common Mistakes

- Saying BeanFactory is commonly used ✗
👉 It's outdated in modern apps

"ApplicationContext is preferred because it provides enterprise features like AOP, events, and eager initialization."

✅ Summary (What Interviewer Expects 🚨)

You MUST be able to explain:

- ✅ What is Spring
 - ✅ IoC vs DI
 - ✅ Types of DI
 - ✅ Role of Spring Container
 - ✅ ApplicationContext vs BeanFactory
-

👉 Next Step

If you want, I'll continue with:

🔥 PART 1 3 — Spring Beans & Scopes (VERY IMPORTANT 🚨)

This includes:

- Bean lifecycle (asked frequently)
- All scopes (including tricky ones like request/session)
- Real interview traps

Just say:

👉 "Next"



next

✅ PART 1 3 — SPRING BEANS & SCOPES (High Priority 🚨)

This is one of the **most asked + most misunderstood** areas in Spring interviews. Expect **cross-questions** here.

◆ 1. What is a Spring Bean?

✓ Definition

A Spring Bean is:

Any object that is **created, managed, and configured** by the Spring Container

✓ How a Bean is Created?

There are 3 common ways:

1. Using Annotations (Most common ✓)

</> Java

```
@Component
class MyService { }
```



Other stereotypes:

- @Service
 - @Repository
 - @Controller
-

2. Using @Bean

</> Java

```
@Configuration
class AppConfig {
    @Bean
    public MyService myService() {
        return new MyService();
    }
}
```



3. XML (Old, rarely used)

</> XML



```
<bean id="myService" class="com.example.MyService"/>
```

✓ Real-world Use Case

- UserService → business logic
- UserRepository → DB access
- PaymentService → external API

All these are **beans managed by Spring**

✓ When to Use Bean?

- Always for:
 - Services
 - Controllers
 - Repositories
 - Config classes
-

✗ When NOT?

- Simple POJOs (DTOs, models)
 - Temporary objects
-

⚠ Common Mistakes

- ✗ Thinking every class is a bean
 - ✗ Forgetting component scanning
 - ✗ Using `new` keyword instead of Spring
-

🎯 Interview Tip

"A Spring Bean is an object managed by the Spring IoC container."

◆ 2. Bean Lifecycle (VERY IMPORTANT 🚨)

✓ Complete Flow

1. Bean Instantiation
2. Dependency Injection
3. BeanNameAware (optional)
4. BeanPostProcessor (before init)
5. Initialization (init method)
6. BeanPostProcessor (after init)
7. Ready to use
8. Destruction (destroy method)



✓ Code Example

`</> Java`



```
@Component
class MyBean {

    @PostConstruct
    public void init() {
        System.out.println("Bean initialized");
    }

    @PreDestroy
    public void destroy() {
        System.out.println("Bean destroyed");
    }
}
```

✓ Real-world Use Case

- Initialize DB connection
- Load cache
- Close resources on shutdown

✓ When to Use Lifecycle?

- Resource management
- Logging
- Initialization logic

⚠ Common Mistakes

- ❌ Ignoring lifecycle completely
- ❌ Using constructor for heavy logic
- ❌ Forgetting destroy methods

🎯 Interview Tip

If asked:

"Explain Bean Lifecycle"

👉 Say:

"Spring first creates the bean, injects dependencies, runs initialization callbacks, and finally calls destroy methods before shutdown."

◆ 3. Bean Scopes (VERY IMPORTANT 🚨)

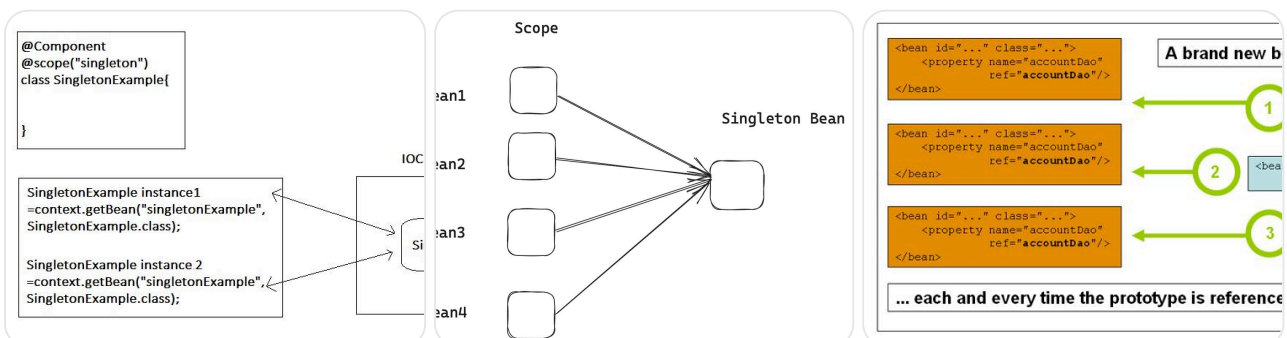
✔ What is Scope?

Scope defines:

How many instances of a bean are created

🔥 All Bean Scopes Explained

◆ 1. Singleton (DEFAULT ✔)



✔ Definition

👉 Only ONE instance per Spring container

✓ Example

</> Java



```
@Component
@Scope("singleton") // default
class MyService { }
```

✓ Behavior

- Shared across entire app
 - Same object everywhere
-

✓ Use Case

- Services
 - Repositories
 - Config classes
-

✗ When NOT?

- Stateful objects
-

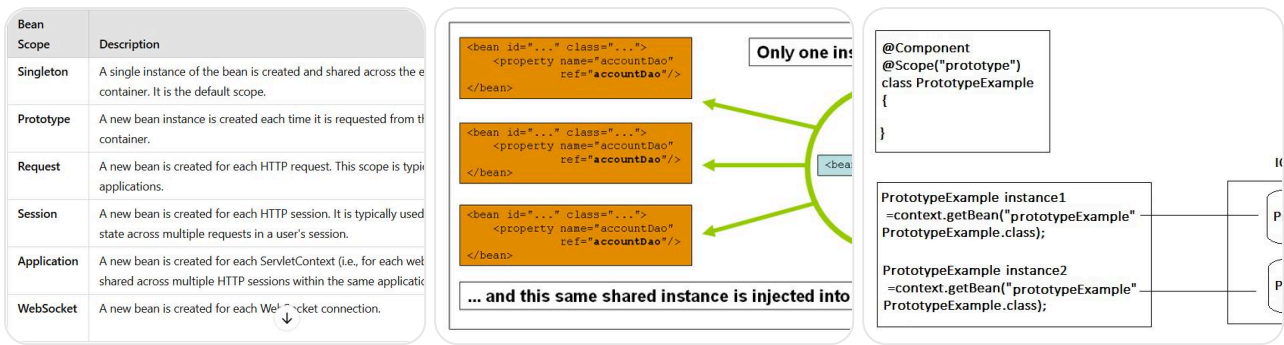
⚠ Mistake

- Assuming new object is created each time ✗
-

🎯 Interview Tip

"Singleton is the default scope in Spring."

◆ 2. Prototype



✓ Definition

👉 New object **every time requested**

✓ Example

</> Java

```

@Component
@Scope("prototype")
class MyBean { }

```



✓ Behavior

- `getBean()` → new instance
- No full lifecycle management (destroy not called)

✓ Use Case

- Temporary objects
- Stateful components

✗ When NOT?

- Heavy objects (performance issue)

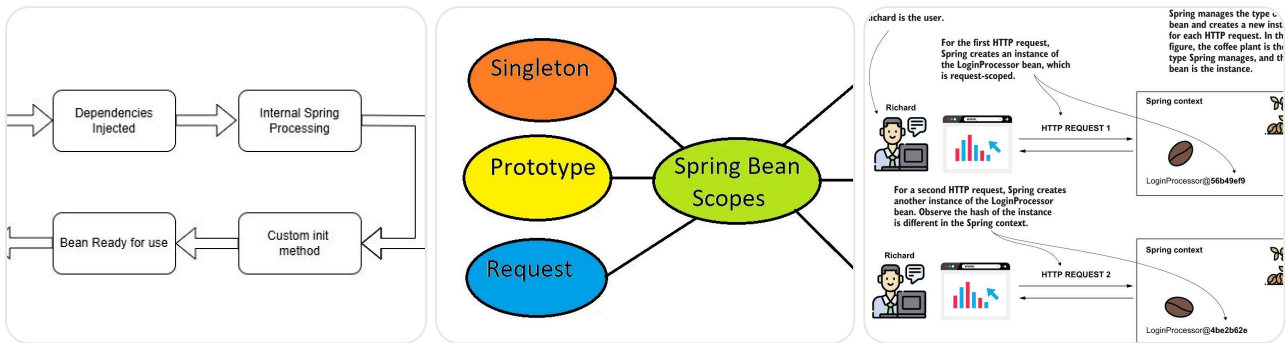
⚠ Mistakes

- Expecting Spring to destroy prototype beans ✗

🎯 Interview Tip

"Spring manages creation but not destruction of prototype beans."

◆ 3. Request Scope (Web Apps)



✓ Definition

👉 One bean per HTTP request

✓ Example

</> Java

```
@Component
@Scope(value = WebApplicationContext.SCOPE_REQUEST)
class RequestBean { }
```

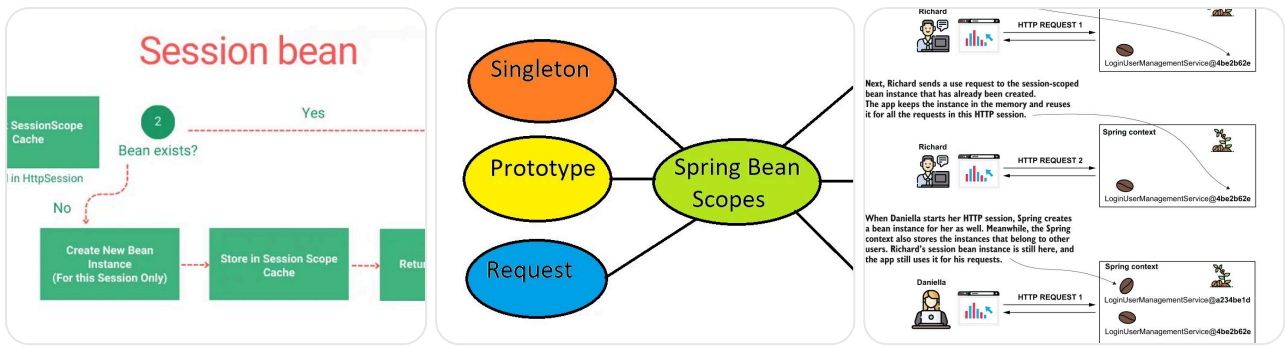
✓ Use Case

- User request-specific data

✗ When NOT?

- Non-web applications

◆ 4. Session Scope



✓ Definition

👉 One bean per user session

✓ Use Case

- Shopping cart
- User login data

⚠ Risk

- Memory leaks if session is long

◆ 5. Application Scope (Global)

✓ Definition

👉 One bean per ServletContext

✓ Use Case

- Shared global data

◆ 6. WebSocket Scope

✓ Definition

👉 One bean per WebSocket session

✓ Use Case

- Real-time apps (chat apps, live feeds)
-

◆ 4. Default Bean Scope

✓ Answer

👉 Singleton

🎯 Interview Trap 🚨

If interviewer asks:

"What is default scope?"

👉 Don't say:

✗ "Prototype"

👉 Correct:

✓ "Singleton"

◆ 5. Real Interview Scenarios

? Q: Singleton bean inside Prototype bean?

👉 Prototype will get:

- Same singleton instance
-

? Q: Prototype inside Singleton?

👉 Only ONE instance injected (important trap 🚨)

Solution:

</> Java



```
@Lookup
public MyPrototypeBean getBean() {
    return null;
}
```

◆ 6. Common Developer Mistakes 🚨

- ❌ Using Singleton for user-specific data
- ❌ Not understanding scope behavior
- ❌ Memory leaks with session scope
- ❌ Wrong assumptions about prototype lifecycle

◆ 7. Quick Revision Table

Scope	Instances	Use Case
Singleton	One	Services
Prototype	Many	Temporary objects
Request	Per request	API data
Session	Per user	Login/cart
Application	Global	Shared data

🎯 Final Interview Summary

You must confidently answer:

- ✔ What is a bean
- ✔ How beans are created
- ✔ Bean lifecycle steps
- ✔ All scopes + use cases
- ✔ Default scope
- ✔ Singleton vs Prototype

Next Step

Next we'll cover:

PART — Dependency Injection Annotations (CRITICAL)

Includes:

- `@Autowired` deep internals
 - Multiple beans problem
 - `@Qualifier` vs `@Primary` (VERY commonly asked)
 - Constructor vs Setter vs Field (interview favorite)
-

Just say:

 "Next"



next

PART — DEPENDENCY INJECTION ANNOTATIONS (CRITICAL)

This section is **extremely important for interviews** because it tests:

- Your understanding of DI internals
 - How Spring resolves ambiguity
 - Clean coding practices
-

1. `@Autowired`

Definition

`@Autowired` is used to:

Automatically inject dependencies **by type** from the Spring container

✓ Example

</> Java



```
@Service
class UserService {

    @Autowired
    private UserRepository userRepository;
}
```

✓ How It Works Internally

1. Spring scans for beans
2. Finds dependency by **type**
3. Injects matching bean

✓ Injection Types with @Autowired

1. Constructor Injection (BEST ✓)

</> Java



```
@Service
class UserService {

    private final UserRepository repo;

    @Autowired // optional in newer Spring
    public UserService(UserRepository repo) {
        this.repo = repo;
    }
}
```

2. Setter Injection

</> Java



```
@Autowired
public void setRepo(UserRepository repo) {
    this.repo = repo;
}
```

3. Field Injection ❌ (avoid)

</> Java



```
@Autowired  
private UserRepository repo;
```

✓ Real-world Use Case

- Injecting repository into service
- Injecting service into controller

✓ When to Use?

- Always for injecting Spring-managed beans

❌ When NOT?

- Don't use for:
 - Primitive values (use @Value)
 - Manual object creation

⚠️ Common Mistakes

- ❌ Using field injection
- ❌ Not understanding type-based injection
- ❌ Ignoring constructor injection best practice

🎯 Interview Tip

"@Autowired performs dependency injection by type and is preferred with constructor injection."

◆ 2. Multiple Beans Problem 🚨

✓ Problem

What happens if **multiple beans of same type** exist?

</> Java



```
@Component
class MySQLRepo implements Repo {}

@Component
class MongoRepo implements Repo {}
```

</> Java



```
@Autowired
Repo repo; // ✗ ERROR
```

✓ Exception

👉 NoUniqueBeanDefinitionException

✓ Why?

Spring finds:

- 👉 2 beans of type Repo
- 👉 Doesn't know which one to inject

◆ 3. NoUniqueBeanDefinitionException

✓ Definition

Occurs when:

Multiple beans of same type exist and Spring cannot decide which one to inject

✓ Example

</> Java



```
@Autowired  
Repo repo; // ❌ ambiguous
```

✓ Fix Required 🚨

Use:

- @Qualifier
- @Primary

◆ 4. @Qualifier

✓ Definition

@Qualifier tells Spring:

Exactly which bean to inject

✓ Example

</> Java



```
@Component("mysqlRepo")  
class MySQLRepo implements Repo {}  
  
@Component("mongoRepo")  
class MongoRepo implements Repo {}
```

</> Java



```
@Autowired  
@Qualifier("mysqlRepo")  
Repo repo;
```

✓ Use Case

- Multiple implementations
 - Strategy pattern
-

✓ When to Use?

- Whenever ambiguity exists
-

✗ When NOT?

- Single bean case (not needed)
-

⚠ Mistakes

- ✗ Wrong bean name
 - ✗ Forgetting qualifier
-

🎯 Interview Tip

"@Qualifier resolves ambiguity by specifying the exact bean name."

◆ 5. @Primary

✓ Definition

@Primary tells Spring:

This is the **default bean** when multiple exist

✓ Example

⌕ Java



```
@Component
@Primary
class MySQLRepo implements Repo {}
```

```
@Component
class MongoRepo implements Repo {}
```

</> Java



```
@Autowired
Repo repo; // injects MySQLRepo
```

✓ Use Case

- Default implementation

✓ When to Use?

- One main implementation + others optional

✗ When NOT?

- When you need dynamic selection → use `@Qualifier`

⚠ Mistakes

- ✗ Using multiple `@Primary` beans
- ✗ Mixing `@Primary` + wrong expectations

🎯 Interview Tip

"`@Primary` defines the default bean, while `@Qualifier` gives explicit control."

◆ 6. Constructor vs Setter vs Field Injection

🔥 Comparison Table

Type	Recommended	Why
Constructor	✅ BEST	Immutable, testable
Setter	⚠️ OK	Optional dependency
Field	❌ BAD	Hard to test

✅ Constructor Injection (Best Practice)

</> Java



```
@Service
class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

✅ Advantages

- Immutable dependencies
- Easy unit testing
- No null issues

✅ Setter Injection

</> Java



```
@Autowired
public void setPaymentService(PaymentService ps) {
    this.paymentService = ps;
}
```

✅ Use Case

- Optional dependencies

✗ Field Injection

</> Java



```
@Autowired
private PaymentService ps;
```

✗ Problems

- Not testable
- Reflection-based injection
- Hard to debug

◆ 7. Real-world Scenario

Problem:

You have:

- Payment via Stripe
- Payment via Razorpay

Solution:

</> Java



```
@Component("stripe")
class StripeService implements PaymentService {}

@Component("razorpay")
class RazorpayService implements PaymentService {}
```

</> Java



```
@Autowired
@Qualifier("stripe")
```

```
PaymentService paymentService;
```

◆ 8. Advanced Interview Traps 🚨

? What if no bean found?

👉 Exception:

NoSuchBeanDefinitionException

? Can @Autowired be optional?

```
</> Java
```

```
@Autowired(required = false)
```



? Can we inject list of beans?

```
</> Java
```

```
@Autowired  
List<PaymentService> services;
```



◆ 9. Common Developer Mistakes 🚨

- ❌ Using field injection everywhere
 - ❌ Ignoring constructor injection
 - ❌ Not handling multiple beans
 - ❌ Overusing @Primary
-

◆ 10. Final Interview Summary

You MUST confidently answer:

- ✓ How @Autowired works
 - ✓ What happens with multiple beans
 - ✓ @Qualifier vs @Primary
 - ✓ Constructor vs Setter vs Field
 - ✓ Common exceptions
-

Perfect Interview Answer (High Value)

If asked:

"How does Spring resolve multiple beans?"

 Answer:

"Spring injects dependencies by type. If multiple beans exist, it throws NoUniqueBeanDefinitionException. We resolve this using @Qualifier for explicit selection or @Primary for default selection."

Next Step

Next is:

PART 1 5 — SPRING BOOT ANNOTATIONS (VERY IMPORTANT)

Includes:

- @SpringBootApplication deep breakdown
 - Auto-configuration internals
 - @Component vs @Service vs @Repository
 - @Bean vs @Component (VERY tricky )
 - Layered architecture
-

Say:

 "Next"

✅ PART 1 5 — SPRING BOOT ANNOTATIONS (VERY IMPORTANT 🚨)

This section is **core Spring Boot internals**. Interviewers often go beyond definitions and test:

- How auto-configuration actually works
- When to use which annotation
- Clean architecture understanding

◆ 1. @SpringBootApplication (MOST IMPORTANT 🚨)

✓ Definition

@SpringBootApplication is a **meta-annotation** that combines:

</> Java



```
@Configuration
@EnableAutoConfiguration
@ComponentScan
```

✓ Example

</> Java



```
@SpringBootApplication
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}
```

✓ What Happens Internally?

When you run the app:

1. Spring starts container
 2. Scans components
 3. Applies auto-configurations
 4. Creates beans
 5. Starts embedded server
-

✓ Real-world Use Case

- Bootstrapping any Spring Boot application
 - Entry point of microservices
-

⚠ Common Mistakes

- ✗ Placing main class in wrong package (breaks scanning)
 - ✗ Not understanding what it includes internally
-

🎯 Interview Tip

"@SpringBootApplication is a combination of configuration, component scanning, and auto-configuration."

◆ 2. @Configuration

✓ Definition

Marks a class as:

Source of bean definitions

✓ Example

</> Java



```
@Configuration
class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new StripeService();
    }
}
```

✓ Use Case

- Manual bean creation
- Third-party class configuration

✗ When NOT?

- For simple components → use `@Component`

⚠ Mistake

- ✗ Confusing `@Configuration` with `@Component`

◆ 3. `@EnableAutoConfiguration`

✓ Definition

Automatically configures:

Beans based on classpath + properties

✓ Example

If you add:

```
</> XML
```

```
spring-boot-starter-web
```



Spring auto-configures:

- Tomcat
 - DispatcherServlet
 - Jackson
-

✓ How It Works (Important 🚨)

Uses:

👉 `spring.factories` / `AutoConfiguration.imports`

Spring checks:

- Dependencies present?
- Conditions satisfied?

Then:

👉 Loads configuration classes automatically

✓ Example

🔗 Java



```
@ConditionalOnClass(DataSource.class)
```

✓ Use Case

- Zero configuration setup
 - Faster development
-

✗ When NOT?

- When you need full manual control
-

⚠ Mistakes

- ✗ Blindly relying on auto-config
 - ✗ Not knowing what is configured
-

Interview Tip

"Spring Boot auto-configures beans based on classpath dependencies and conditional annotations."

◆ 4. Custom Auto Configuration (Advanced)

✓ When Needed?

- Creating reusable libraries
 - Building frameworks
-

✓ Steps

1. Create config class
 2. Use `@Configuration`
 3. Add conditions
 4. Register in auto-config file
-

✓ Example

 Java 

```
@Configuration
@ConditionalOnClass(MyService.class)
class MyAutoConfig {
    @Bean
    public MyService myService() {
        return new MyService();
    }
}
```

✓ Use Case

- Internal frameworks
 - Company-wide reusable modules
-

◆ 5. Excluding Auto Configuration

✓ Why?

Sometimes auto-config:

- Conflicts
 - Not needed
-

✓ Example

```
</> Java

@SpringBootApplication(
    exclude = {DataSourceAutoConfiguration.class}
)
```



✓ Use Case

- No DB in project
 - Custom configuration required
-

⚠ Mistake

- ✗ Excluding without understanding impact
-

◆ 6. @Component vs @Service vs @Repository

✓ All are Stereotypes

Annotation	Layer
@Component	Generic
@Service	Business logic
@Repository	Data access

✓ Example

</> Java



```
@Service
class UserService {}

@Repository
class UserRepository {}

@Component
class Utility {}
```

✓ Key Difference

👉 Behavior:

- @Repository → Exception translation (important 🚨)
- @Service → Semantic meaning

✓ Use Case

- Follow layered architecture

✗ Mistake

- ✗ Using @Component everywhere

🎯 Interview Tip

"@Service and @Repository are specialized forms of @Component with semantic meaning."

◆ 7. @Bean vs @Component (VERY IMPORTANT 🚨)

✓ Difference Table

Feature	@Component	@Bean
Usage	Class level	Method level
Control	Less	Full control
Use case	Your classes	Third-party classes

✓ Example

@Component

</> Java



```
@Component
class MyService {}
```

@Bean

</> Java



```
@Configuration
class Config {

    @Bean
    public MyService myService() {
        return new MyService();
    }
}
```

✓ When to Use?

Use @Component when:

- You own the class

Use @Bean when:

- Third-party class
- Need custom initialization

⚠️ Common Mistakes

- ❌ Using @Component for third-party classes
 - ❌ Not understanding lifecycle difference
-

🎯 Interview Tip

"@Component is for auto-detection, while @Bean gives explicit control over bean creation."

◆ 8. Stereotype Annotations

✓ Definition

Annotations that indicate:

👉 Role of a class

✓ Types

- @Component
 - @Service
 - @Repository
 - @Controller
-

✓ Use Case

- Improves readability
 - Enables AOP features
-

◆ 9. Layered Architecture (VERY IMPORTANT 🚨)

✓ Standard Layers



✓ Example Flow

</> Java



```
@RestController
class UserController {

    @Autowired
    UserService service;
}
```

</> Java



```
@Service
class UserService {

    @Autowired
    UserRepository repo;
}
```

</> Java



```
@Repository
class UserRepository {}
```

✓ Why Important?

- Separation of concerns
- Clean code
- Easy testing

✗ Mistakes

- ✗ Writing DB logic in controller
- ✗ Skipping service layer

🎯 Interview Tip

"Layered architecture separates concerns and improves maintainability."

◆ 10. Real-world Interview Scenarios 🚨

? What if component scan fails?

👉 Bean not created → injection fails

? Can we have multiple config classes?

👉 YES ✅

? Can @Bean override @Component?

👉 YES (depends on context)

◆ 11. Common Developer Mistakes 🚨

- ❌ Not understanding auto-config
 - ❌ Wrong package structure
 - ❌ Using @Component everywhere
 - ❌ Ignoring layered architecture
-

◆ 12. Final Interview Summary

You MUST confidently explain:

- ✅ @SpringBootApplication internals
- ✅ Auto-configuration working
- ✅ @Component vs @Service vs @Repository
- ✅ @Bean vs @Component

-  Layered architecture
-

Perfect Interview Answer

If asked:

"What does @SpringBootApplication do?"

 Answer:

"It combines @Configuration, @EnableAutoConfiguration, and @ComponentScan to bootstrap the application, automatically configure beans, and scan components."

Next Step

Next is:

PART 1 6 — SPRING BOOT CONFIGURATION

Includes:

- application.properties vs YAML
 - Profiles (VERY important )
 - Changing ports
 - Embedded servers (Tomcat, Jetty, Undertow)
 - Advantages & disadvantages
-

Say:

 "Next"



next

PART 1 6 — SPRING BOOT CONFIGURATION (VERY IMPORTANT)

This section is **practical** + **frequently asked**. Interviewers expect you to know:

- How configuration works internally
- How to switch environments (dev/prod)
- How Spring Boot auto-configures servers

◆ 1. application.properties VS application.yml

✔ What is Configuration?

Spring Boot uses:

External configuration to manage application behavior without changing code

✔ application.properties

⟨⟩ properties



```
server.port=8080
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
```

✔ application.yml

⟨⟩ YAML



```
server:
  port: 8080

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mydb
    username: root
```

✔ Key Difference

Feature	properties	YAML
Syntax	key=value	hierarchical

Feature	properties	YAML
Readability	Medium	High
Complex config	Hard	Easy

✓ When to Use?

- Small config → `.properties`
- Complex/nested config → `.yaml` ✓

✗ When NOT?

- YAML → avoid if team not familiar

⚠ Common Mistakes

- ✗ Wrong indentation in YAML
- ✗ Mixing both formats incorrectly

🎯 Interview Tip

"YAML is preferred for hierarchical configuration due to better readability."

◆ 2. `pom.xml` (VERY IMPORTANT 🚨)

✓ What is it?

👉 Maven configuration file that manages:

- Dependencies
- Build lifecycle
- Plugins

✓ Example

</> XML



```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
</dependencies>
```

✓ Key Concept

👉 Starter Dependencies

Example:

- spring-boot-starter-web
- spring-boot-starter-data-jpa

They include:

- Required libraries automatically

✓ Use Case

- Dependency management
- Version compatibility

⚠ Mistakes

- ✗ Adding conflicting dependencies
- ✗ Not using starter dependencies

🎯 Interview Tip

"Spring Boot starters simplify dependency management by providing pre-configured dependencies."

◆ 3. Changing Server Port

✓ Default Port

👉 8080

✓ Change to 8090

```
</> properties  
  
server.port=8090
```



✓ Use Case

- Running multiple services
 - Avoid port conflicts
-

⚠ Mistake

- ❌ Port already in use
-

🎯 Interview Tip

"Server port can be changed using server.port property."

◆ 4. Profiles (VERY IMPORTANT 🚨)

✓ What are Profiles?

Profiles allow:

Different configurations for different environments

✓ Example

```
</> properties
```



```
spring.profiles.active=dev
```

✓ Profile-specific files

```
application-dev.properties  
application-prod.properties
```



✓ Using @Profile

</> Java

```
@Profile("dev")  
@Bean  
public DataSource devDb() { }
```



✓ Real-world Use Case

Environment	DB
dev	Local
prod	Cloud

✓ When to Use?

- Always in real applications

✗ When NOT?

- Small apps

⚠ Mistakes

- ✗ Hardcoding environment values
- ✗ Not using profiles

Interview Tip

"Profiles help manage environment-specific configurations like dev, test, and production."

◆ 5. Default Embedded Server

✓ Answer

👉 Apache Tomcat

✓ How it Works

Spring Boot:

- Embeds server inside application
 - No external deployment needed
-

✓ Example

⌞ Java



```
SpringApplication.run(App.class, args);
```

👉 Starts Tomcat internally

✓ Use Case

- Standalone applications
 - Microservices
-

Interview Tip

"Spring Boot uses embedded Tomcat by default."

◆ 6. Jetty & Undertow Support

✓ Alternatives to Tomcat

- Jetty
 - Undertow
-

✓ How to Use Jetty

</> XML



```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jetty</artifactId>
</dependency>
```

✓ When to Use?

Server	Use Case
Tomcat	Default
Jetty	Lightweight
Undertow	High performance

⚠ Mistake

- ✗ Forgetting to exclude Tomcat
-

🎯 Interview Tip

"Spring Boot supports multiple embedded servers like Tomcat, Jetty, and Undertow."

◆ 7. Standalone Spring Boot Application

✔ Definition

👉 App that runs independently with embedded server

✔ Example

</> Java



```
public static void main(String[] args) {  
    SpringApplication.run(App.class, args);  
}
```

✔ Why Important?

- No WAR deployment
 - No external server needed
-

✔ Use Case

- Microservices
 - APIs
-

🎯 Interview Tip

"Spring Boot applications are standalone and run with embedded servers."

◆ 8. Advantages of Spring Boot

✔ Key Advantages

- ☒ Auto-configuration
- ☒ Embedded server
- ☒ Faster development
- ☒ Microservices-ready
- ☒ Minimal configuration

✓ Real-world Impact

- Build APIs in minutes
 - Reduce boilerplate
-

◆ 9. Disadvantages of Spring Boot

✗ Key Disadvantages

- ✗ Less control over configuration
 - ✗ Higher memory usage
 - ✗ Debugging auto-config can be tricky
-

✓ When NOT to Use?

- Low-level performance-critical apps
 - Simple scripts
-

◆ 10. Real Interview Scenarios 🚨

? How to change config without restarting?

👉 Use:

- Spring Cloud Config
 - Environment variables
-

? Can we override properties?

👉 YES ✓

Priority:

Command Line > Environment > application.properties



◆ 11. Common Developer Mistakes 🚨

- ❌ Hardcoding configs
 - ❌ Not using profiles
 - ❌ Ignoring YAML indentation
 - ❌ Dependency conflicts
-

◆ 12. Final Interview Summary

You MUST confidently explain:

- ✔ Properties vs YAML
 - ✔ Profiles (VERY important 🚨)
 - ✔ Embedded servers
 - ✔ Port configuration
 - ✔ Advantages & disadvantages
-

🎯 Perfect Interview Answer

If asked:

"What are Spring Boot profiles?"

👉 Answer:

"Profiles allow environment-specific configuration by separating settings for dev, test, and production using profile-specific property files."

👉 Next Step

Next is:

🔥 **PART 1 7 — REST API & WEB (HIGH PRIORITY 🚨)**

Includes:

- REST principles
- `@RestController` vs `@Controller`

- HTTP methods & idempotency
 - Request/Response annotations
 - XML vs JSON
 - REST vs SOAP
-

Say:

👉 "Next"



next

✅ PART 1 7 — REST API & WEB (HIGH PRIORITY 🚨)

This is **one of the most asked sections**. Interviewers expect:

- Concept clarity (REST principles)
 - Annotation-level understanding
 - HTTP semantics (idempotency, safety)
 - Real-world API design
-

◆ 1. REST Principles

✓ What is REST?

REST = **Representational State Transfer**

It is an **architectural style** for designing scalable web services.

✓ Core Principles

1. Stateless

👉 Each request is independent

- Server does NOT store client state
- Client sends all required data

2. Client-Server Separation

- Frontend and backend are independent
-

3. Uniform Interface

- Standard HTTP methods
 - Consistent URL structure
-

4. Resource-Based

- Everything is a resource

Example:

```
/users  
/orders  
/products
```



✓ Real-world Use Case

- E-commerce APIs
 - Banking APIs
 - Mobile app backends
-

✗ Common Mistakes

- ✗ Using verbs in URL (/getUsers)
 - ✗ Not following HTTP methods correctly
-

🎯 Interview Tip

"REST is a stateless, resource-based architecture using HTTP methods."

◆ 2. @Controller vs @RestController

✔ Difference

Feature	@Controller	@RestController
Use case	MVC (views)	REST APIs
Return type	View (HTML)	JSON/XML
@ResponseBody needed?	YES	NO

✔ Example

@Controller

</> Java



```
@Controller
class MyController {

    @GetMapping("/home")
    public String home() {
        return "home"; // returns HTML page
    }
}
```

@RestController

</> Java



```
@RestController
class UserController {

    @GetMapping("/user")
    public String getUser() {
        return "Pratham";
    }
}
```

✔ Key Concept

👉 @RestController = @Controller + @ResponseBody

Interview Tip

" @RestController returns data directly, while @Controller returns views."

◆ 3. Default Return Type in @RestController

✓ Answer

👉 JSON (via Jackson)

✓ Example

</> Java



```
@GetMapping("/user")
public User getUser() {
    return new User("Pratham", 21);
}
```

👉 Response:

</> JSON



```
{
  "name": "Pratham",
  "age": 21
}
```

⚠ Mistake

- ❌ Thinking it returns String only
-

◆ 4. Why @ResponseBody is Required?

✓ Explanation

Without it:

👉 Spring tries to resolve **view name**

With it:

👉 Returns **data directly**

✓ Example

</> Java



```
@Controller
class TestController {

    @ResponseBody
    @GetMapping("/data")
    public String data() {
        return "Hello";
    }
}
```

◆ 5. REST vs SOAP

✓ Comparison

Feature	REST	SOAP
Protocol	HTTP	XML-based
Format	JSON/XML	XML only
Performance	Fast	Slow
Flexibility	High	Strict

✓ Use Case

- REST → Modern apps
 - SOAP → Legacy enterprise systems
-

Interview Tip

"REST is lightweight and flexible, while SOAP is protocol-heavy and secure."

◆ 6. Safe vs Idempotent Requests

✓ Safe Methods

👉 Do NOT modify data

- GET
 - HEAD
-

✓ Idempotent Methods

👉 Same result no matter how many times called

- GET
 - PUT
 - DELETE
-

! NOT Idempotent

- POST
-

✓ Example

`</> http`



`PUT /user/1 → same result every time`

`POST /user → creates new user each time`

Interview Tip

"GET is safe and idempotent, POST is neither, PUT is idempotent."

◆ 7. HTTP Methods (CRUD Mapping)

✓ Mapping Table

Operation	HTTP Method	Annotation
Create	POST	@PostMapping
Read	GET	@GetMapping
Update	PUT	@PutMapping
Delete	DELETE	@DeleteMapping

✓ Example

</> Java



```
@RestController
@RequestMapping("/users")
class UserController {

    @GetMapping
    public List<User> getUsers() {}

    @PostMapping
    public User createUser(@RequestBody User user) {}

    @PutMapping("/{id}")
    public User updateUser(@PathVariable int id) {}

    @DeleteMapping("/{id}")
    public void deleteUser(@PathVariable int id) {}
}
```

⚠ Mistakes

- ❌ Using GET for updates
- ❌ Using POST for everything

◆ 8. @RequestMapping

✓ Definition

Maps HTTP requests to handler methods

✓ Example

</> Java



```
@RequestMapping(value="/users", method=RequestMethod.GET)
```

✓ Shortcut Annotations

- @GetMapping
 - @PostMapping
-

🎯 Interview Tip

" @RequestMapping is the base mapping annotation for HTTP requests."

◆ 9. @PathVariable

✓ Definition

Extracts value from URL

✓ Example

</> Java



```
@GetMapping("/users/{id}")  
public User getUser(@PathVariable int id) {}
```

✓ URL

/users/101



◆ 10. @RequestParam

✓ Definition

Extracts query parameters

✓ Example

</> Java

```
@GetMapping("/users")
public User getUser(@RequestParam int id) {}
```



✓ URL

/users?id=101



✓ Difference

Feature	PathVariable	RequestParam
Location	URL path	Query string

◆ 11. @RequestBody

✓ Definition

✓ Example

```
</> Java📄  
  
@PostMapping("/users")  
public User create(@RequestBody User user) {}
```

✓ Input JSON

```
</> JSON📄  
  
{  
  "name": "Pratham"  
}
```

⚠ Mistake

- ✗ Forgetting @RequestBody → null object
-

◆ 12. consumes & produces

✓ Definition

Defines:

- Input format (consumes)
 - Output format (produces)
-

✓ Example

```
</> Java📄  
  
@PostMapping(  
  value = "/users",  
  consumes = "application/json",
```

```
    produces = "application/xml"  
  )
```

✓ Use Case

- API contracts
- Versioning

◆ 13. Returning XML from REST API

✓ How?

Add dependency:

</> XML

```
jackson-dataformat-xml
```



✓ Example

</> Java

```
@GetMapping(value="/user", produces="application/xml")  
public User getUser() {}
```



✓ Output

</> XML

```
<User>  
  <name>Pratham</name>  
</User>
```



⚠ Mistake

- ❌ Missing XML dependency

◆ 14. Real-world API Flow 🚨

Client → Controller → Service → Repository → DB



◆ 15. Common Developer Mistakes 🚨

- ❌ Not following REST naming conventions
 - ❌ Mixing GET and POST
 - ❌ Ignoring idempotency
 - ❌ Not validating request body
 - ❌ Poor URL design
-

◆ 16. Final Interview Summary

You MUST confidently explain:

- ✔ REST principles
 - ✔ HTTP methods & idempotency
 - ✔ Controller annotations
 - ✔ Request handling annotations
 - ✔ JSON vs XML
-

🎯 Perfect Interview Answer

If asked:

"What is REST?"

👉 Answer:

"REST is a stateless architectural style that uses HTTP methods to perform operations on resources represented typically in JSON format."

Next Step

Next is:

PART 1 8 — EXCEPTION HANDLING IN SPRING BOOT (VERY IMPORTANT)

Includes:

- `@ExceptionHandler`
 - `@ControllerAdvice`
 - Global exception handling
 - Real-world error flow
-

Say:

 "Next"



next

PART 1 8 — EXCEPTION HANDLING IN SPRING BOOT (VERY IMPORTANT)

This is a **high-impact interview topic** because it shows:

- Your API design maturity
 - How cleanly you handle failures
 - Whether your system is production-ready
-

◆ 1. Why Exception Handling is Important?

✓ Problem Without Exception Handling

</> Java



```
@GetMapping("/user/{id}")
public User getUser(@PathVariable int id) {
    return service.getUser(id); // may throw exception
}
```

👉 If error occurs:

- Stack trace returned ❌
- Poor user experience ❌
- Security risk ❌

✓ Goal

👉 Return clean, structured error responses

✓ Example Response

</> JSON



```
{
  "timestamp": "2026-05-02",
  "status": 404,
  "message": "User not found"
}
```

◆ 2. @ExceptionHandler

✓ Definition

Handles exceptions:

At controller level

✓ Example

</> Java



```
@RestController
class UserController {

    @ExceptionHandler(RuntimeException.class)
    public String handleException(RuntimeException ex) {
        return "Error: " + ex.getMessage();
    }
}
```

✓ Flow

Exception → Controller → @ExceptionHandler → Response



✓ Use Case

- Small applications
- Controller-specific errors

✗ When NOT?

- Large applications → use global handling

⚠ Mistakes

- ✗ Duplicating handlers in multiple controllers
- ✗ Catching generic Exception everywhere

🎯 Interview Tip

"@ExceptionHandler handles exceptions locally within a controller."

◆ 3. @ControllerAdvice (VERY IMPORTANT 🚨)

✔ Definition

Provides **global exception handling** across all controllers

✔ Example

</> Java



```
@ControllerAdvice
class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<String> handleUserNotFound(UserNotFoundException ex) {
        return ResponseEntity
            .status(404)
            .body(ex.getMessage());
    }
}
```

✔ Why Needed?

- Centralized logic
 - Clean architecture
 - Avoid duplication
-

✔ Real-world Use Case

- Handling:
 - Validation errors
 - DB errors
 - API errors
-

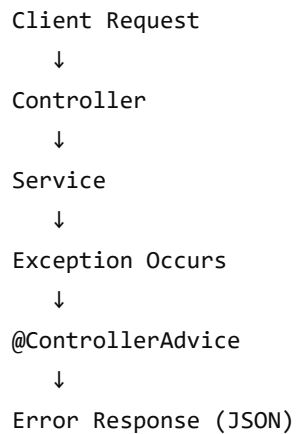
🎯 Interview Tip

"@ControllerAdvice is used for global exception handling across all controllers."

◆ 4. Global Exception Handling Flow 🚨

✓ Flow Diagram

```
graph TD; A[Client Request] --> B[Controller]; B --> C[Service]; C --> D[Exception Occurs]; D --> E["@ControllerAdvice"]; E --> F[Error Response (JSON)];
```



Client Request

↓

Controller

↓

Service

↓

Exception Occurs

↓

@ControllerAdvice

↓

Error Response (JSON)



✓ Best Practice

Always return:

- Status code
- Message
- Timestamp
- Error details

✓ Example Response Class

</> Java

```
class ErrorResponse {  
    private String message;  
    private int status;  
    private LocalDateTime timestamp;  
}
```



◆ 5. REST Exception Handling (Best Practice 🚨)

✓ Use @RestControllerAdvice

👉 Combination of:

- `@ControllerAdvice`
- `@ResponseBody`

✓ Example

</> Java



```
@RestControllerAdvice
class GlobalHandler {

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handle(Exception ex) {

        ErrorResponse error = new ErrorResponse(
            ex.getMessage(),
            500,
            LocalDateTime.now()
        );

        return new ResponseEntity<>(error, HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

◆ 6. Custom Exceptions (VERY IMPORTANT 🚨)

✓ Example

</> Java



```
class UserNotFoundException extends RuntimeException {
    public UserNotFoundException(String msg) {
        super(msg);
    }
}
```

✓ Usage

</> Java



```
if(user == null) {  
    throw new UserNotFoundException("User not found");  
}
```

✓ Why Important?

- Clean error handling
- Better readability
- Domain-specific errors

◆ 7. HTTP Status Codes

✓ Common Codes

Code	Meaning
200	OK
201	Created
400	Bad Request
404	Not Found
500	Internal Server Error

✓ Example

</> Java



```
return ResponseEntity.status(HttpStatus.NOT_FOUND).body("User not found");
```

◆ 8. Real-world Example (Complete Flow 🚨)

✓ Controller

⌌ Java



```
@GetMapping("/user/{id}")
public User getUser(@PathVariable int id) {
    return service.findUser(id);
}
```

✓ Service

⌌ Java



```
public User findUser(int id) {
    return repo.findById(id)
        .orElseThrow(() -> new UserNotFoundException("User not found"));
}
```

✓ Global Handler

⌌ Java



```
@ExceptionHandler(UserNotFoundException.class)
public ResponseEntity<?> handle(UserNotFoundException ex) {
    return ResponseEntity.status(404).body(ex.getMessage());
}
```

◆ 9. Advanced Concepts (Interview Boost 🚨)

✓ Multiple Exception Handlers

⌌ Java



```
@ExceptionHandler({NullPointerException.class, IllegalArgumentException.class})
```

✓ Handling Validation Errors

</> Java



```
@ExceptionHandler(MethodArgumentNotValidException.class)
```

✓ Returning Structured Errors

👉 Always prefer DTO over String

◆ 10. Common Developer Mistakes 🚨

- ❌ Returning raw exceptions
- ❌ Not using global handler
- ❌ Using generic Exception everywhere
- ❌ Not setting proper HTTP status
- ❌ Poor error response format

◆ 11. Final Interview Summary

You MUST confidently explain:

- ✓ @ExceptionHandler
- ✓ @ControllerAdvice
- ✓ Global handling flow
- ✓ Custom exceptions
- ✓ HTTP status codes

Perfect Interview Answer

If asked:

"How do you handle exceptions in Spring Boot?"

 Answer:

"We use `@ControllerAdvice` for global exception handling and `@ExceptionHandler` to map exceptions to proper HTTP responses with structured error messages."

Next Step

Next is:

 **PART 1 9** — SPRING DATA JPA & HIBERNATE (VERY IMPORTANT )

Includes:

- Hibernate vs JPA
 - Lazy loading
 - Caching
 - `@Entity`, `@Id`, etc.
 - `CrudRepository` vs `JpaRepository`
 - DB connectivity
-

Say:

 "Next"



next

 **PART 1 9** — SPRING DATA JPA & HIBERNATE (VERY IMPORTANT )

This is **core backend + DB integration**, and interviewers often go deep into:

- ORM concepts

- Performance (lazy loading, caching)
 - Repository abstraction
 - SQL vs ORM differences
-

◆ 1. What is Hibernate?

✓ Definition

Hibernate is an **ORM (Object Relational Mapping)** framework that:

Maps Java objects → Database tables

✓ Problem Without Hibernate

⟨⟩ Java



```
Connection con = DriverManager.getConnection(...);
Statement st = con.createStatement();
ResultSet rs = st.executeQuery("SELECT * FROM users");
```

👉 Problems:

- Boilerplate code ❌
 - Error-prone ❌
 - Hard to maintain ❌
-

✓ With Hibernate

⟨⟩ Java



```
User user = entityManager.find(User.class, 1);
```

👉 Clean + abstraction ✅

✓ Real-world Use Case

- Banking systems
- E-commerce backend

- Enterprise apps
-

✓ Advantages of Hibernate

- ✓ No need to write SQL
 - ✓ Automatic table mapping
 - ✓ Caching support
 - ✓ Database independence
-

✗ When NOT to Use?

- High-performance raw SQL systems
 - Complex queries (sometimes better with native SQL)
-

⚠ Mistakes

- ✗ Blindly trusting ORM (without understanding SQL)
 - ✗ Not optimizing queries
-

🎯 Interview Tip

"Hibernate is an ORM framework that maps Java objects to database tables and eliminates boilerplate JDBC code."

◆ 2. Hibernate vs JPA

✓ Key Difference

Feature	Hibernate	JPA
Type	Implementation	Specification
Vendor	Hibernate	Standard
Usage	Direct	Through implementation

✓ Analogy

👉 JPA = Interface

👉 Hibernate = Implementation

✓ Example

⌘ Java

```
@Entity  
class User { }
```



👉 This is JPA annotation

👉 Hibernate implements it

✓ Use Case

- Always code using **JPA**
 - Hibernate runs underneath
-

🎯 Interview Tip

"JPA is a specification, and Hibernate is its most popular implementation."

◆ 3. Lazy Loading (VERY IMPORTANT 🚨)

✓ Definition

Data is loaded **only when needed**

✓ Example

⌘ Java

```
@OneToMany(fetch = FetchType.LAZY)
```



```
List<Order> orders;
```

✓ Behavior

- User loaded first
- Orders loaded only when accessed

✓ Real-world Use Case

- Large datasets
- Performance optimization

✗ Eager Loading

</> Java



```
@OneToMany(fetch = FetchType.EAGER)
```

👉 Loads everything immediately

⚠ Common Mistakes

- ✗ LazyInitializationException
- ✗ Using LAZY outside transaction

🎯 Interview Tip

"Lazy loading improves performance by fetching data only when required."

◆ 4. Caching in Hibernate

✓ Types of Cache

1. First-Level Cache (Default)

- Session-level
 - Always enabled
-

2. Second-Level Cache

- Application-level
 - Requires configuration
-

✓ Example

⌌ Java



```
User u1 = session.get(User.class, 1);  
User u2 = session.get(User.class, 1);
```

👉 Second call → cache hit

✓ Use Case

- Frequently accessed data
-

⚠ Mistakes

- ❌ Not configuring cache properly
 - ❌ Overusing cache
-

🎯 Interview Tip

"Hibernate uses first-level cache by default and supports second-level cache for performance optimization."

◆ 5. JPA Annotations (CORE 🚨)

✓ @Entity

</> Java



```
@Entity
class User { }
```

👉 Maps class → table

✓ @Table

</> Java



```
@Table(name = "users")
```

👉 Custom table name

✓ @Id

</> Java



```
@Id
private Long id;
```

👉 Primary key

✓ Example

</> Java



```
@Entity
@Table(name = "users")
class User {

    @Id
    private Long id;

    private String name;
}
```

⚠ Mistakes

- ❌ Missing @Id
- ❌ Wrong table mapping

◆ 6. Database Connectivity in Spring Boot

✓ Configuration

⌘ properties



```
spring.datasource.url=jdbc:mysql://localhost:3306/db
spring.datasource.username=root
spring.datasource.password=1234
```

✓ What Happens Internally?

- DataSource created
- Connection pool initialized
- Hibernate connected

✓ Default Pool

👉 HikariCP

🎯 Interview Tip

"Spring Boot auto-configures DataSource and uses HikariCP by default."

◆ 7. Statement vs PreparedStatement

✓ Difference

Feature	Statement	PreparedStatement
Performance	Low	High

Feature	Statement	PreparedStatement
Security	✗ SQL Injection	✓ Safe
Compilation	Every time	Once

✓ Example

⌄ Java



```
PreparedStatement ps = con.prepareStatement(
    "SELECT * FROM users WHERE id=?"
);
ps.setInt(1, 1);
```

🎯 Interview Tip

"PreparedStatement prevents SQL injection and improves performance."

◆ 8. CrudRepository vs JpaRepository (VERY IMPORTANT 🚨)

✓ Difference

Feature	CrudRepository	JpaRepository
Basic CRUD	✓	✓
Pagination	✗	✓
Sorting	✗	✓
JPA features	Limited	Full

✓ Example

</> Java



```
public interface UserRepo extends JpaRepository<User, Long> {  
}
```

✓ When to Use?

👉 Always prefer **JpaRepository** ✓

⚠ Mistake

- ✗ Using CrudRepository in modern apps
-

🎯 Interview Tip

"JpaRepository extends CrudRepository and provides additional features like pagination and sorting."

◆ 9. Real-world Flow 🚨

Controller → Service → Repository → JPA → Hibernate → DB



◆ 10. Common Developer Mistakes 🚨

- ✗ Not understanding lazy loading
 - ✗ Writing inefficient queries
 - ✗ Ignoring indexes
 - ✗ Using EAGER unnecessarily
 - ✗ Not handling transactions
-

◆ 11. Final Interview Summary

You MUST confidently explain:

- ✓ Hibernate vs JPA
 - ✓ Lazy vs Eager loading
 - ✓ Caching
 - ✓ JPA annotations
 - ✓ Repository differences
 - ✓ DB configuration
-

Perfect Interview Answer

If asked:

"What is the difference between Hibernate and JPA?"

👉 Answer:

"JPA is a specification for ORM, while Hibernate is an implementation of JPA that provides actual functionality."

Next Step

Next is:

PART 2 0 — ACTUATOR & MONITORING

Includes:

- Spring Boot Actuator
 - /health , /metrics , /info
 - Production monitoring
-

Say:

👉 "Next"



✓ PART 2 0 — SPRING BOOT ACTUATOR & MONITORING

This section is **very important for production systems**. Interviewers use it to check:

- Do you understand **observability**?
- Can you monitor and debug applications in production?

◆ 1. What is Spring Boot Actuator?

✓ Definition

Spring Boot Actuator provides:

Production-ready features to **monitor and manage applications**

✓ What It Gives You

- Application health
 - Metrics (CPU, memory, requests)
 - Environment properties
 - Beans info
 - Logging details
-

✓ Dependency

</> XML



```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

✓ Real-world Use Case

- Monitor microservices health
 - Debug production issues
 - Integrate with tools like Prometheus, Grafana
-

✗ When NOT to Use?

- Very small apps (not required)
-

⚠ Mistake

- ✗ Exposing all endpoints publicly (security risk 🚨)
-

🎯 Interview Tip

"Spring Boot Actuator provides production-ready monitoring and management endpoints."

◆ 2. Actuator Endpoints (CORE 🚨)

✓ Common Endpoints

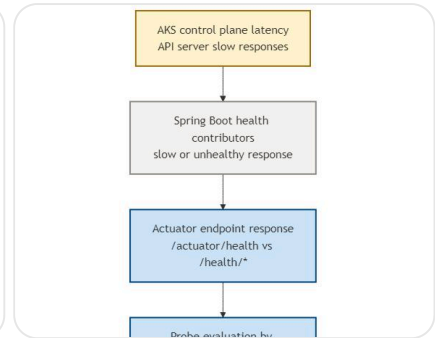
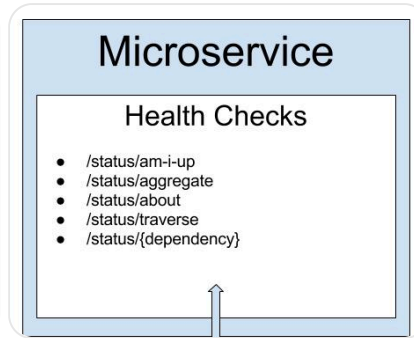
Endpoint	Purpose
/actuator/health	App status
/actuator/metrics	Performance metrics
/actuator/info	App info
/actuator/beans	All beans
/actuator/env	Environment properties

◆ /health (MOST IMPORTANT 🚨)

```
demo git:(master) x http localhost:8080/actuator/health
HTTP/1.1 200
Content-Type: application/vnd.spring-boot.actuator.v2+json;charset=UTF-8
Date: Sun, 11 Mar 2018 21:19:37 GMT
Transfer-Encoding: chunked

{
  "details": {
    "db": {
      "details": {
        "database": "H2",
        "hello": 1
      },
      "status": "UP"
    },
    "diskSpace": {
      "details": {
        "free": 336497074176,
        "threshold": 10485760,
        "total": 499963174912
      },
      "status": "UP"
    },
    "memory": {

```



✓ Example Response

</> JSON



```
{
  "status": "UP"
}
```

✓ Use Case

- Load balancer checks
- Kubernetes liveness probe

✗ Status Values

- UP
- DOWN
- OUT_OF_SERVICE

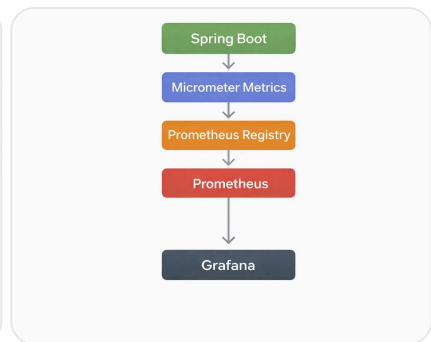
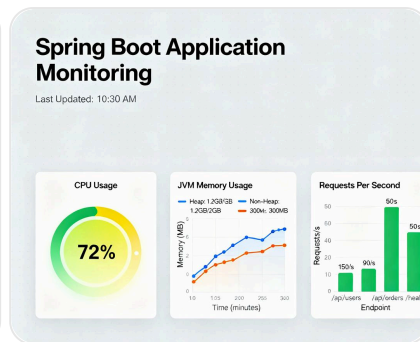
🎯 Interview Tip

" /health endpoint is used to check application status."

◆ /metrics

```
// 20201029233324
// http://127.0.0.1:8080/actuator/metrics/http.server.requests

{
  "name": "http.server.requests",
  "description": null,
  "baseUnit": "seconds",
  "measurements": [{}],
  "availableTags": [
    {
      "tag": "uri",
      "values": [
        "/actuator/metrics/{requiredMetricName}",
        "/push-log",
        "/actuator/metrics"
      ]
    },
    {
      "tag": "demo-tag",
      "values": [
        "demo-tag"
      ]
    }
  ]
}
```



✓ What It Shows

- HTTP requests
- JVM memory
- CPU usage

✓ Example

/actuator/metrics/http.server.requests



✓ Use Case

- Performance monitoring
- Bottleneck detection

🎯 Interview Tip

" /metrics provides application performance data using Micrometer."

◆ /info

✓ Example

<> properties

```
info.app.name=MyApp
info.app.version=1.0
```



✓ Response

</> JSON



```
{
  "app": {
    "name": "MyApp",
    "version": "1.0"
  }
}
```

✓ Use Case

- Version tracking
- Build info

◆ /beans

✓ What It Shows

- All Spring beans
- Dependencies

✓ Use Case

- Debugging DI issues

◆ /env

✓ Shows

- Environment variables
 - Config properties
-

Risk

- Sensitive data exposure 

◆ 3. Enabling Actuator Endpoints

✓ By Default

Only `/health` is exposed

✓ Enable All

 properties

```
management.endpoints.web.exposure.include=*
```



✓ Enable Specific

 properties

```
management.endpoints.web.exposure.include=health,info,metrics
```



Security Best Practice

 properties

```
management.endpoint.health.show-details=never
```



◆ 4. Custom Health Indicator

✓ Example

</> Java



```
@Component
class CustomHealth implements HealthIndicator {

    @Override
    public Health health() {
        if(serviceUp()) {
            return Health.up().build();
        } else {
            return Health.down().withDetail("error", "Service down").build();
        }
    }
}
```

✓ Use Case

- DB check
- External API health

◆ 5. Monitoring Tools Integration

✓ Common Tools

- Prometheus
- Grafana
- ELK Stack

✓ Flow

Spring Boot → Actuator → Micrometer → Prometheus → Grafana



◆ 6. Real-world Scenario

Example:

You deployed microservice

Problem:

- API slow

👉 Use:

- `/metrics` → check latency
 - `/health` → check dependencies
-
-

◆ 7. Common Developer Mistakes 🚨

- ❌ Exposing all endpoints publicly
 - ❌ Not securing actuator endpoints
 - ❌ Ignoring monitoring in production
 - ❌ Not customizing health checks
-
-

◆ 8. Final Interview Summary

You MUST confidently explain:

- ✔ What is Actuator
 - ✔ Key endpoints
 - ✔ Monitoring flow
 - ✔ Security considerations
 - ✔ Custom health checks
-

🎯 Perfect Interview Answer

If asked:

"What is Spring Boot Actuator?"

👉 Answer:

"Spring Boot Actuator provides production-ready endpoints like health, metrics, and info to monitor and manage applications."

👉 Next Step

Next is:

🔥 PART 2 1 — MICROSERVICES ARCHITECTURE (VERY IMPORTANT 🚨)

Includes:

- Monolithic vs Microservices
 - Load balancing
 - Circuit breaker
 - Spring Cloud basics
-

Say:

👉 "Next"



next

✅ PART 2 1 — MICROSERVICES ARCHITECTURE (VERY IMPORTANT 🚨)

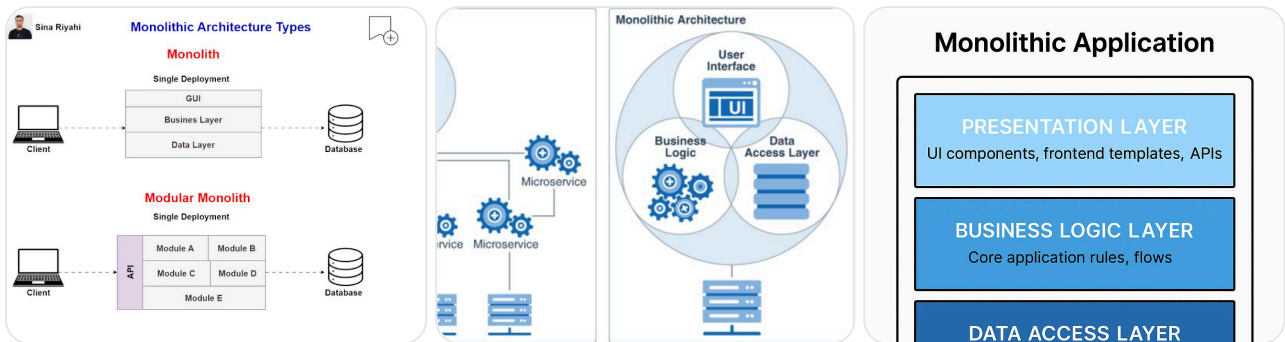
This topic is **heavily asked in interviews**, especially for backend and Spring Boot roles.

Interviewers evaluate:

- System design thinking
 - Scalability understanding
 - Real-world architecture decisions
-

◆ 1. Monolithic vs Microservices

◆ Monolithic Architecture



✓ Definition

👉 Entire application is built as **one single unit**

✓ Example

- One app contains:
 - User module
 - Payment module
 - Order module

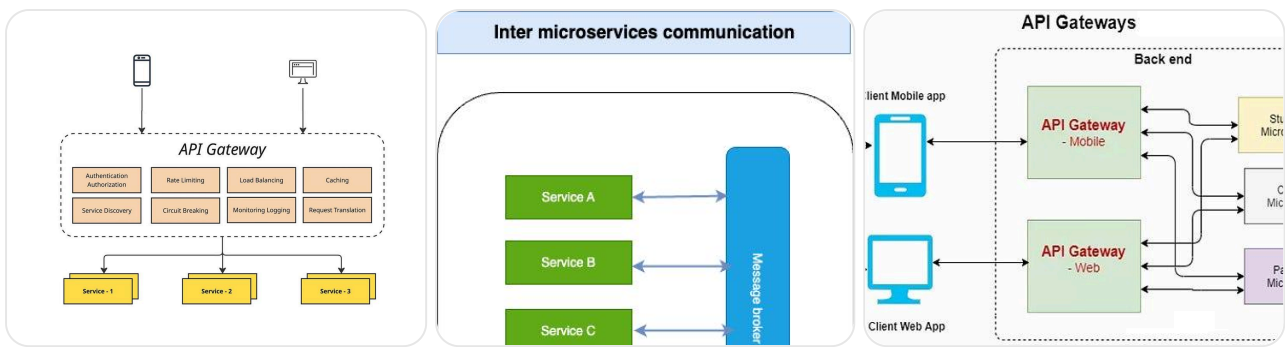
✓ Advantages

- Simple to develop
- Easy deployment

✗ Disadvantages

- Hard to scale
- Tight coupling
- Difficult maintenance

◆ Microservices Architecture



✓ Definition

👉 Application is divided into **small independent services**

✓ Example

- User Service
- Payment Service
- Order Service

Each runs independently

✓ Advantages

- Independent deployment
- Scalability
- Fault isolation

✗ Disadvantages

- Complex system
- Network overhead
- Debugging difficulty

🎯 Interview Tip

"Monolithic apps are single units, while microservices split functionality into independent services."

◆ 2. Advantages & Disadvantages (Direct Question 🚨)

✓ Advantages of Microservices

- ✓ Scalability
 - ✓ Independent deployment
 - ✓ Technology flexibility
 - ✓ Fault isolation
-

✗ Disadvantages

- ✗ Complex communication
 - ✗ Data consistency issues
 - ✗ Deployment complexity
-
-

◆ 3. Load Balancing

✓ Definition

Distributes incoming requests across multiple instances

✓ Types

- Client-side (Ribbon - deprecated)
 - Server-side (NGINX, Load balancer)
-

✓ Example

User Request → Load Balancer → Service Instance 1 / 2 / 3



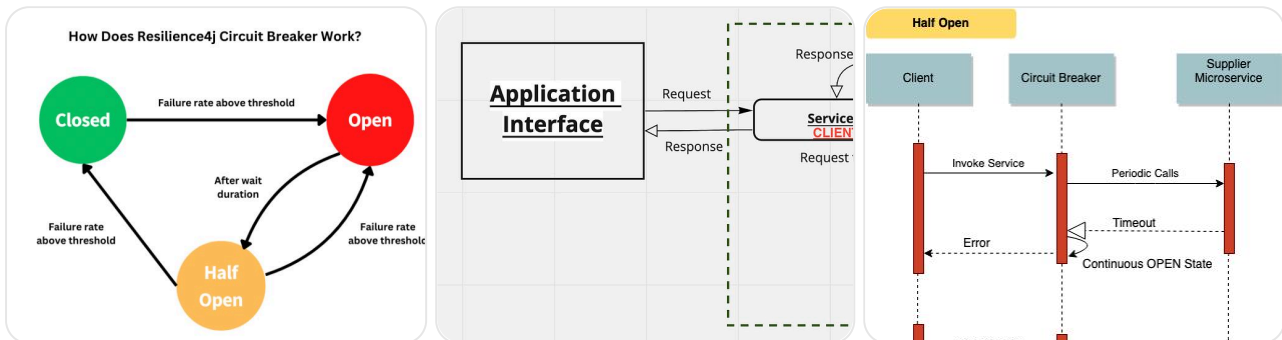
✓ Use Case

- High traffic applications

🎯 Interview Tip

"Load balancing improves availability and scalability by distributing traffic."

◆ 4. Circuit Breaker (VERY IMPORTANT 🚨)



✓ Definition

Prevents system failure when a service is down

✓ States

- Closed → normal
- Open → calls blocked
- Half-open → testing

✓ Example

If Payment Service fails:

- 👉 Circuit opens → stop calling it
- 👉 Return fallback response

✓ Tools

- Resilience4j
- Hystrix (deprecated)

✓ Use Case

- Fault tolerance
-

🎯 Interview Tip

"Circuit breaker prevents cascading failures by stopping calls to failing services."

◆ 5. Role of Spring Boot in Microservices

✓ Why Spring Boot?

- Lightweight
 - Fast startup
 - Easy REST APIs
 - Embedded server
-

✓ Example

Each microservice:

- Built using Spring Boot
 - Exposes REST endpoints
-

✓ Use Case

- Microservices development
-
-

◆ 6. Spring Cloud Overview

✓ What is Spring Cloud?

Provides tools to build **distributed systems**

✓ Key Components

Tool	Purpose
Eureka	Service discovery
Config Server	Central config
Gateway	API routing
Feign Client	REST calls
Resilience4j	Fault tolerance

✓ Example Flow

Client → API Gateway → Microservices → DB



◆ 7. Real-world Architecture

✓ Example: E-commerce

User Service → Order Service → Payment Service



✓ Communication Types

- REST (synchronous)
 - Kafka (asynchronous)
-
-

◆ 8. Common Developer Mistakes

-
- ❌ Converting monolith → microservices too early
 - ❌ Ignoring network latency
 - ❌ Not handling failures
 - ❌ Poor service boundaries
-
-

◆ 9. Interview Scenarios 🚨

? When to use microservices?

👉 When:

- Application is large
 - Needs scalability
 - Multiple teams working
-

? When NOT?

👉 When:

- Small app
 - Simple logic
-
-

◆ 10. Final Interview Summary

You MUST confidently explain:

- ✅ Monolith vs Microservices
 - ✅ Load balancing
 - ✅ Circuit breaker
 - ✅ Spring Cloud basics
 - ✅ Real-world architecture
-

Perfect Interview Answer

If asked:

"What is microservices architecture?"

 Answer:

"Microservices architecture divides an application into small independent services that can be developed, deployed, and scaled independently."

Next Step

Next is:

PART 2 2 — SECURITY & ASYNC (IMPORTANT)

Includes:

- Spring Security basics
 - Authentication vs Authorization
 - @Async
 - Multithreading use cases
-

Say:

 "Next"



next

PART 2 2 — SECURITY & ASYNC (IMPORTANT)

This section tests whether you understand:

- Application security fundamentals (very common interview topic)
 - Concurrency & performance optimization using async processing
-

◆ 1. Spring Security Basics

✓ What is Spring Security?

A framework that provides **authentication and authorization** for Java applications

✓ What it Handles

- Login / logout
 - Password encryption
 - Role-based access
 - Protection against attacks (CSRF, XSS)
-

✓ Dependency

</> XML



```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

✓ Default Behavior 🚨

When added:

- All endpoints are **secured**
 - Default username: user
 - Password: generated at runtime
-

⚠ Common Mistake

- ❌ Forgetting that security is auto-enabled
-

🎯 Interview Tip

"Spring Security provides authentication and authorization mechanisms to secure applications."

◆ 2. Authentication vs Authorization (VERY IMPORTANT 🚨)

✓ Authentication

👉 Who are you?

- Login process
 - Username + password
-

✓ Authorization

👉 What can you access?

- Roles & permissions
-

✓ Example

Step	Meaning
Login	Authentication
Access /admin	Authorization

🎯 Interview Tip

"Authentication verifies identity, while authorization determines access rights."

◆ 3. Basic Security Configuration

✓ Example

</> Java



```
@Configuration
@EnableWebSecurity
class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public").permitAll()
                .anyRequest().authenticated()
            )
            .formLogin();

        return http.build();
    }
}
```

✓ Behavior

- /public → accessible
- Other endpoints → secured

✓ Use Case

- Role-based APIs
- Login systems

◆ 4. Password Encoding

✓ Why Needed?

👉 Never store plain passwords ❌

✓ Example

</> Java



```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

✓ Storage

\$2a\$10\$abcdef...



🎯 Interview Tip

"Passwords should be stored using hashing algorithms like BCrypt."

◆ 5. @Async (VERY IMPORTANT 🚨)

✓ Definition

Runs a method **asynchronously** (in a separate thread)

✓ Enable Async

</> Java



```
@EnableAsync
@SpringBootApplication
class App {}
```

✓ Example

</> Java



```
@Async
public void sendEmail() {
    // runs in background
}
```

✓ Behavior

- Main thread continues
- Task runs in background

✓ Real-world Use Case

- Sending emails
- Notifications
- Background processing

✗ When NOT?

- Critical synchronous operations

⚠ Mistakes

- ✗ Calling async method inside same class (won't work 🚨)
- ✗ Ignoring thread pool configuration

🎯 Interview Tip

"@Async allows non-blocking execution of methods in separate threads."

◆ 6. Return Types in @Async

✓ Supported Types

⌌ Java



```
@Async
public CompletableFuture<String> process() {
    return CompletableFuture.completedFuture("Done");
}
```

✓ Use Case

- Parallel processing
- Combining results

◆ 7. When & Why Async is Needed

✓ Problem Without Async

Request → Process → Send Email → Response



👉 Slow response ❌

✓ With Async

Request → Process → Response



Email (async)



👉 Fast response ✅

◆ 8. Threading Concept (Interview Angle 🚨)

✓ Without Async

- Single thread
- Blocking

✓ With Async

- Multi-threaded
 - Non-blocking
-
-

◆ 9. Common Developer Mistakes 🚨

Security

- ✗ Storing plain passwords
 - ✗ Not securing endpoints
 - ✗ Ignoring authorization
-

Async

- ✗ Not enabling `@EnableAsync`
 - ✗ Using async for DB transactions incorrectly
 - ✗ Not handling exceptions in async
-
-

◆ 10. Real-world Scenario 🚨

Example:

User registers → system sends email

👉 Without async:

- Slow response

👉 With async:

- Immediate response + background email
-
-

◆ 11. Final Interview Summary

You MUST confidently explain:

- ✓ Spring Security basics
 - ✓ Authentication vs Authorization
 - ✓ Password encoding
 - ✓ @Async usage
 - ✓ Async flow
-

Perfect Interview Answer

If asked:

"What is @Async?"

👉 Answer:

"@Async allows methods to run in a separate thread asynchronously, improving performance by making operations non-blocking."

👉 Next Step

Final section:

PART 2 3 — EXTERNAL INTEGRATIONS

Includes:

- Calling third-party APIs
 - RestTemplate vs WebClient (VERY IMPORTANT 🚨)
-

Say:

👉 "Next"

     ...

next

✓ PART 2 3 — EXTERNAL INTEGRATIONS (VERY IMPORTANT 🚨)

This is **practical + frequently asked** because almost every real system:

- Calls payment APIs
 - Uses third-party services
 - Integrates microservices
-

◆ 1. Interacting with Third-Party APIs

✓ What is it?

Calling external services from your application using HTTP

✓ Examples

- Payment gateway (Razorpay, Stripe)
 - Weather API
 - Google Maps API
-

✓ Real-world Flow

Client → Your Service → Third-party API → Response → Client



✓ Example Use Case

👉 Your app calls:

- Payment API → process payment
 - SMS API → send OTP
-

✓ Key Challenges

- Network latency
- API failures

- Timeout handling
 - Rate limiting
-

Common Mistakes

-  No timeout handling
 -  Not handling failures
 -  Blocking calls in high-traffic apps
-

Interview Tip

"External API integration involves calling third-party services using HTTP clients like RestTemplate or WebClient."

◆ 2. Accessing Third-Party URLs

✓ Two Main Approaches

Tool	Type
RestTemplate	Synchronous (blocking)
WebClient	Asynchronous (non-blocking)

◆ 3. RestTemplate (OLD BUT ASKED)

✓ Definition

A synchronous HTTP client used to call REST APIs

✓ Example

</> Java



```
@RestController
class MyController {

    @GetMapping("/data")
    public String getData() {

        RestTemplate restTemplate = new RestTemplate();

        String response = restTemplate.getForObject(
            "https://api.example.com/data",
            String.class
        );

        return response;
    }
}
```

✓ Supported Methods

- GET
- POST
- PUT
- DELETE

✓ Behavior

Request → RestTemplate → External API → Wait → Response



👉 Blocking call

✓ Use Case

- Simple applications
- Low traffic

✗ When NOT?

- High-performance systems
- Reactive applications

⚠ Mistakes

- ❌ Using RestTemplate in new projects
 - ❌ Not handling errors
-

🎯 Interview Tip

"RestTemplate is a synchronous HTTP client and is now deprecated in favor of WebClient."

◆ 4. WebClient (MODERN APPROACH 🚨)

✓ Definition

A non-blocking, reactive HTTP client introduced in Spring WebFlux

✓ Example

🔗 Java



```
@Service
class ApiService {

    private final WebClient webClient =
        WebClient.create("https://api.example.com");

    public String getData() {

        return webClient.get()
            .uri("/data")
            .retrieve()
            .bodyToMono(String.class)
            .block(); // blocking here for simplicity

    }
}
```

✓ Non-blocking Example

⌞ Java



```
public Mono<String> getData() {  
    return webClient.get()  
        .uri("/data")  
        .retrieve()  
        .bodyToMono(String.class);  
}
```

✓ Behavior

Request → WebClient → Async Call → Continue Execution → Response later



✓ Advantages

- Non-blocking
- High performance
- Better for microservices

✓ Use Case

- High traffic systems
- Reactive apps
- Microservices communication

✗ When NOT?

- Simple apps (overkill)

⚠ Mistakes

- ✗ Using `.block()` everywhere (kills async benefit 🚨)
- ✗ Not understanding reactive flow

Interview Tip

"WebClient is a non-blocking, reactive alternative to RestTemplate and is recommended for modern applications."

◆ 5. RestTemplate vs WebClient (VERY IMPORTANT)

✓ Comparison Table

Feature	RestTemplate	WebClient
Type	Blocking	Non-blocking
Performance	Lower	Higher
Thread usage	One thread per request	Event loop
Status	Deprecated	Recommended

◆ 6. Error Handling in API Calls

✓ Example (WebClient)

</> Java 

```
webClient.get()
    .uri("/data")
    .retrieve()
    .onStatus(
        status -> status.is4xxClientError(),
        response -> Mono.error(new RuntimeException("Client error"))
    )
```

✓ Use Case

- Handle API failures gracefully
-
-

◆ 7. Real-world Scenario

Example: Payment API

User → Order Service → Payment API → Response



Problems:

- API slow
 - API down
-

Solutions:

- Retry
 - Circuit breaker
 - Timeout
-
-

◆ 8. Best Practices

- ✓ Use WebClient for new apps
 - ✓ Add timeout configuration
 - ✓ Handle exceptions properly
 - ✓ Use circuit breaker for resilience
-
-

◆ 9. Common Developer Mistakes

- ❌ Using blocking calls in microservices
 - ❌ Ignoring retries
 - ❌ Not validating API response
 - ❌ Hardcoding URLs
-
-

◆ 10. Final Interview Summary

You MUST confidently explain:

- ✔ Third-party API integration
 - ✔ RestTemplate vs WebClient
 - ✔ Blocking vs non-blocking
 - ✔ Error handling
 - ✔ Real-world use cases
-

🎯 Perfect Interview Answer

If asked:

"RestTemplate vs WebClient?"

👉 Answer:

"RestTemplate is a synchronous blocking client, while WebClient is a non-blocking reactive client and is preferred for modern, high-performance applications."

✅ COMPLETE SYLLABUS DONE 🚀

You now have:

- 🔥 Deep understanding
 - 🔥 Interview-ready answers
 - 🔥 Real-world use cases
-

👉 What Next?

If you want next level prep, I can give you:

🔥 MOCK INTERVIEW (REAL QUESTIONS + FOLLOW UPS)

🔥 CODING + DESIGN QUESTIONS (Spring Boot)

🔥 PROJECT-BASED QUESTIONS (VERY IMPORTANT 🚨)

🔥 RAPID REVISION NOTES (1-page cheatsheet)

Just tell me:

👉 "Mock Interview" or "Revision Notes"



2 4 Coding Programs (Must Prepare 🚨)

- Character occurrence in string
- Reverse string
- Anagram check
- Prime number program
- Second highest salary using streams

✅ PART 2 4 — CODING PROGRAMS (Must Prepare 🚨)

These are **very frequently asked** in interviews—especially to test:

- Java fundamentals
- String manipulation
- Stream API
- Logical thinking

I'll cover each with:

- ✓ Approach
- ✓ Optimized solution
- ✓ Edge cases
- ✓ Interview tips

◆ 1. Character Occurrence in String

✓ Problem

Count frequency of each character in a string

✓ Approach

- Traverse string
 - Store count in HashMap
-

✓ Java Code (Optimized)

</> Java



```
import java.util.*;

public class CharCount {
    public static void main(String[] args) {
        String str = "programming";

        Map<Character, Integer> map = new HashMap<>();

        for (char ch : str.toCharArray()) {
            map.put(ch, map.getOrDefault(ch, 0) + 1);
        }

        System.out.println(map);
    }
}
```

✓ Output

```
{p=1, r=2, o=1, g=2, a=1, m=2, i=1, n=1}
```



✓ Using Streams (Advanced 🚨)

</> Java



```
import java.util.*;
import java.util.stream.*;

public class CharCountStream {
    public static void main(String[] args) {
```

```
String str = "programming";

Map<Character, Long> result =
    str.chars()
        .mapToObj(c -> (char) c)
        .collect(Collectors.groupingBy(c -> c, Collectors.counting()));

System.out.println(result);
}
}
```

⚠ Edge Cases

- Empty string
- Case sensitivity (A vs a)

🎯 Interview Tip

Use `getOrDefault()` or Streams for clean code.

◆ 2. Reverse String

✓ Approach 1: Using StringBuilder (BEST)

```
</> Java 

public class ReverseString {
    public static void main(String[] args) {
        String str = "hello";

        String reversed = new StringBuilder(str).reverse().toString();

        System.out.println(reversed);
    }
}
```

✓ Approach 2: Without Built-in (Important 🚨)

</> Java



```
public class ReverseManual {
    public static void main(String[] args) {
        String str = "hello";
        String result = "";

        for (int i = str.length() - 1; i >= 0; i--) {
            result += str.charAt(i);
        }

        System.out.println(result);
    }
}
```

✓ Optimized (Using char array)

</> Java



```
public class ReverseEfficient {
    public static void main(String[] args) {
        char[] arr = "hello".toCharArray();

        int left = 0, right = arr.length - 1;

        while (left < right) {
            char temp = arr[left];
            arr[left] = arr[right];
            arr[right] = temp;

            left++;
            right--;
        }

        System.out.println(new String(arr));
    }
}
```



Mistake

- ✗ Using += inside loop ($O(n^2)$)



Interview Tip

Mention both built-in and manual approach.

◆ 3. Anagram Check

✓ Definition

Two strings are anagrams if:

👉 Same characters, same frequency

✓ Approach 1: Sorting

</> Java



```
import java.util.Arrays;

public class Anagram {
    public static void main(String[] args) {
        String s1 = "listen";
        String s2 = "silent";

        char[] a = s1.toCharArray();
        char[] b = s2.toCharArray();

        Arrays.sort(a);
        Arrays.sort(b);

        System.out.println(Arrays.equals(a, b));
    }
}
```

✓ Approach 2: HashMap (Better 🚀)

</> Java



```
import java.util.*;

public class AnagramMap {
    public static void main(String[] args) {
        String s1 = "listen";
        String s2 = "silent";
```

```
Map<Character, Integer> map = new HashMap<>();

for (char c : s1.toCharArray()) {
    map.put(c, map.getOrDefault(c, 0) + 1);
}

for (char c : s2.toCharArray()) {
    if (!map.containsKey(c)) {
        System.out.println(false);
        return;
    }
    map.put(c, map.get(c) - 1);
    if (map.get(c) == 0) map.remove(c);
}

System.out.println(map.isEmpty());
}
```

⚠ Edge Cases

- Different lengths
- Case sensitivity

🎯 Interview Tip

Sorting is easy, HashMap is optimal ($O(n)$).

◆ 4. Prime Number Program

✓ Definition

Prime = divisible only by 1 and itself

✓ Optimized Approach

👉 Check till $\sqrt{n}$

✓ Java Code

</> Java



```
public class PrimeCheck {
    public static void main(String[] args) {
        int n = 29;

        if (n <= 1) {
            System.out.println("Not Prime");
            return;
        }

        boolean isPrime = true;

        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0) {
                isPrime = false;
                break;
            }
        }

        System.out.println(isPrime ? "Prime" : "Not Prime");
    }
}
```

✓ Time Complexity

👉 $O(\sqrt{n})$

⚠ Mistakes

- ✗ Loop till n
- ✗ Not handling $n \leq 1$

🎯 Interview Tip

Always mention $\sqrt{n}$ optimization 🚀

◆ 5. Second Highest Salary Using Streams (VERY IMPORTANT 🚀)

✓ Problem

Find second highest number in list

✓ Java Code (Streams)

</> Java



```
import java.util.*;

public class SecondHighest {
    public static void main(String[] args) {
        List<Integer> salaries = Arrays.asList(5000, 7000, 8000, 8000, 6000);

        Integer secondHighest = salaries.stream()
            .distinct()
            .sorted(Comparator.reverseOrder())
            .skip(1)
            .findFirst()
            .orElse(null);

        System.out.println(secondHighest);
    }
}
```

✓ Output

7000



✓ Explanation

- `distinct()` → remove duplicates
 - `sorted(desc)` → highest first
 - `skip(1)` → skip highest
 - `findFirst()` → second highest
-

⚠ Edge Cases

- Less than 2 elements

- All values same
-

Interview Tip

Mention why `distinct()` is needed 🚨

BONUS: Without Streams

 Java



```
Collections.sort(salaries);  
System.out.println(salaries.get(salaries.size() - 2));
```

⚠️ Not ideal (duplicates issue)

◆ Final Interview Summary 🚨

You MUST be able to:

- ✓ Write clean Java code
 - ✓ Explain time complexity
 - ✓ Handle edge cases
 - ✓ Provide optimized solution
 - ✓ Use Streams confidently
-

What Interviewer Looks For

- Clean logic
 - Optimization thinking
 - Stream knowledge
 - Edge case handling
-

Next (HIGH VALUE 🚀)

If you want to go next level:

 "Give me 20 coding questions with solutions"

🔥 "Give me Stream API advanced problems"

🔥 "Give me real interview coding test"

Just tell me 👍

